

AI Agent 全栈项目 Harness 工程化系统设计方案

极简 Token 消耗 · 敏捷多 Agent 协同 · 全流程落地方案（以万能视频下载总结器为例）

版本：v1.0

日期：2026-08-15

状态：正式设计方案

一、方案概述与设计哲学

在 AI 编程与 Agent 全栈开发范式中，**Harness（驾驭与控制框架）** 的核心本质并非放任 AI 自由发挥，而是通过 **规则约束、任务编排、上下文架构与反馈闭环**，将人类工程师的精力精准聚焦于需求分析、架构设计与关键节点纠偏等高价值环节。

本方案专门针对**雏形框架开发与全栈项目落地**，提出了一套兼顾**多 Agent 专业化分工与极度节省 Token** 的专业 Harness 工程体系。方案以文档中所提及的“万能视频下载总结器”（集成 YT-DLP、Python 后端、前端界面、SEO/GEO 优化、Stripe 国际支付）为标准案例，提供可复用的工程范式。



图 1: Harness 核心控制哲学与 Token 极简流转架构图

核心 Token 节约机制：

- 路由无 LLM 化：** 使用 Python 本地规则脚本（`subagent_router.py`）解析用户意图，代替大模型全局 Task Allocator 调度。
- 上下文极小化隔离：** 开发新功能时彻底重置对话（Reset Context），单个 Agent 仅载入 `docs/ARCHITECTURE.md` 切片与当前编辑文件，拒绝全量代码库输入。
- 代码逻辑替代 LLM 推理：** 将测试驱动（TDD）、Git 检查点提交等流程写成 Shell/Python 自动化脚本，避免 Agent 盲目尝试消耗 Token。

二、项目文件夹目录架构与作用说明

完整的 Harness 系统采用了模块化、高凝聚的目录结构设计，将控制规则、可复用技能、项目记忆与业务代码严格区分：

```
my-harness-project/
├── .harness/
│   ├── AGENTS.md          # Harness 核心控制配置目录（神经中枢）
│   ├── mcp_config.json    # 主 Agent 全局行为准则与控制边界
│   ├── subagents/         # MCP 工具 (Firecrawl, Context7) 挂载配置
│   │   ├── planner.md     # 子 Agent 独立 Prompt (隔离定义)
│   │   ├── coder.md       # 架构与方案设计 Agent
│   │   └── tester.md      # 核心模块编码 Agent
│   └── tester.md          # 自动化自测与纠偏 Agent
├── .skills/               # 开源与前沿 Skills 提示词与自动化脚本库
│   ├── speckite/         # Speckite 风格：SDD (规范驱动开发) 技能库
│   │   ├── spec_writer.md # 需求拆解与规范文档生成 Prompt
│   │   └── spec_checker.md # 验收标准校验 Prompt
```

```
| └─ superpowers/           # Superpowers 风格：工作流与自动化脚本
|   └─ tdd_runner.sh         # 本地 TDD 测试执行与错误提取脚本
|       └─ subagent_router.py # 本地轻量级 Task 分发/路由脚本（零 Token）
└─ docs/                    # 项目架构与历史记忆库（上下文增量归档）
    └─ ARCHITECTURE.md       # 项目主架构与最新 API 切片文档
    └─ CHECKPOINTS.md        # 回滚检查点与模块状态日志
└─ src/                     # 项目业务源代码（完全受护区）
    └─ backend/              # Python / YT-DLP 后端的核心代码
    └─ frontend/             # 前端界面及优化模块
```

各文件夹及文件的核心作用

目录 / 文件	类型	核心作用与职责说明
<code>.harness/AGENTS.md</code>	规则控制	定义全局开发规范、Plan-First 强制原则、上下文切换条件以及边界限制。
<code>.harness/mcp_config.json</code>	能力扩展	挂载 Firecrawl（网页抓取）与 Context7（最新文档）等 MCP 服务，防止 Agent 使用过时 API。
<code>.harness/subagents/</code>	代理阵列	包含 <code>planner.md</code> 、 <code>coder.md</code> 、 <code>tester.md</code> ，通过 Prompt 分工实现职责单一化。
<code>.skills/speckite/</code>	前沿 Skill	借鉴 SpecKite 开源项目，将需求转化为结构化 SDD 规范文档，明确验收标准。
<code>.skills/superpowers/</code>	前沿 Skill	借鉴 Superpowers 工作流，利用 Python/Shell 脚本实现本地路由与自动化 TDD 校验。
<code>docs/ARCHITECTURE.md</code>	项目记忆	存储增量架构状态，每次开发完成后由 Agent 自动更新，供新对话快速回溯记忆。

三、全流程阶段实践与 Harness 设计图



1. 方案设计阶段 (Planner Agent)

前置需求由工程师梳理（如前端界面、后端配置、YT-DLP 核心能力、Stripe 国际支付）。通过启动 **Planner Agent** 并强制开启 `Plan mode`，AI 进行全网调研后输出规范设计。在工程师回复“APPROVED”前，严禁开启编码，防止 AI 过度设计。

2. 编码开发阶段 (Coder Agent)

为 AI 配置 **Context7 MCP** (技术文档检索) 与 **Firecrawl MCP** (网页抓取)，避免使用过时 API。每完成一个核心功能（如 YT-DLP 核心解析），自动将架构总结写入 `docs/ARCHITECTURE.md` 并提交 Git。开启下一个模块时新建对话，仅载入最新架构切片，防上下文污染。

3. 测试验证阶段 (Tester Agent)

使用 Cursor 内置的 **browser use** 能力实现 UI 全流程自测。当遇到复杂报错（如 B 站视频下载 403 权限卡死、Markdown 渲染久修未愈）时，人工及时介入微调提示词线索；同时可引入 Mock 模拟数据快速测试渲染效果，缩短反馈链路。

4. 功能扩展阶段 (Subagents & Checkpoints)

核心下载功能跑通后，扩展 Markdown 优化、思维导图全屏/下载、字幕文件下载等独立需求时，引导 AI 调用 **subagents** 进行并行开发。每完成一个子功能即更新文档与提交代码，设立 Git 回滚检查点。在处理 SEO/GEO 等关联优化时，直接复用上文环境以实现高效提效。

四、需要的核心代码与提示词 Skill 文件组装

1. 全局主控规则： `.harness/AGENTS.md`

```
# AGENTS.md - Harness 全局控制与 Token 节约准则

## 1. 方案先行 (Plan-First Principle)
- 任何需求启动前，必须由 Planner Agent 输出规范方案 (Spec Document)。
- 方案未获得人类工程师显式确认 ("APPROVED") 前，禁止调用编码工具。

## 2. 上下文隔离与 Token 节省规则
- 编写新功能时，要求新建对话 (Reset Context)。
- 仅允许引入 `.harness/` 定义、`.docs/ARCHITECTURE.md` 以及当前涉及的源代码文件。
- 严禁全量载入项目不相关代码历史。

## 3. 工具使用规范
- 遇到第三方 SDK 或框架 API 查询时，强制调用 Context7/Firecrawl MCP。
- 禁止凭空捏造 API，减少因语法报错重复耗费 Token 的尝试。
```

2. 工具挂载配置： `.harness/mcp_config.json`

```
{
  "mcpServers": {
    "firecrawl": {
      "command": "npx",
      "args": ["-y", "@mendable/firecrawl-mcp-server"],
      "env": { "FIRECRAWL_KEY": "YOUR_API_KEY" },
      "description": "高效抓取最新技术网页，获取准确 API 说明"
    },
    "context7": {
      "command": "uvx",
      "args": ["context7-mcp"],
      "description": "精准检索最新开源技术文档与 SDK 定义"
    }
  }
}
```

3. 前沿 SDD 技能文档： `.skills/speckite/spec_writer.md`

```
# Skill: SDD 规范驱动生成器 (SpecKite Style)

当收到新功能需求时，请按以下结构生成精简规范文档，不要直接写代码：

1. 核心目标：一句话说明新增功能逻辑。
2. 文件变更范围：列出需要新建/修改的文件路径。
3. 验收标准 (Acceptance Criteria):
   - [ ] 验收点 1: ...
   - [ ] 验收点 2: ...

待人类输入 "APPROVED" 后，将此规范转交给 Coder Agent 执行。
```

4. 本地零 Token 路由脚本： `.skills/superpowers/subagent_router.py`

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
import sys
import json

def route_task(user_prompt):
    prompt = user_prompt.lower()

    if any(k in prompt for k in ["plan", "design", "方案", "设计", "架构"]):
        return {
            "target_agent": "planner",
            "context_files": ["docs/ARCHITECTURE.md"],
            "instruction": "使用 SDD 规范输出极简设计方案。"
        }
    elif any(k in prompt for k in ["test", "bug", "403", "fix", "测试", "报错"]):
        return {
            "target_agent": "tester",
            "context_files": ["docs/CHECKPOINTS.md"],
            "instruction": "分析日志，启动单点修复流程。"
        }
    else:
        return {
            "target_agent": "coder",
            "context_files": ["docs/ARCHITECTURE.md"],
            "instruction": "读取 Spec 规范，执行模块局部编码。"
        }

if __name__ == "__main__":
    if len(sys.argv) > 1:
        task = " ".join(sys.argv[1:])
        config = route_task(task)
        print("[Harness Router] 分发配置:")
        print(json.dumps(config, ensure_ascii=False, indent=2))
    else:
        print("用法: python3 subagent_router.py <需求描述>")
```

五、Harness 开源工具对比与标准落地流程

开源工具能力矩阵对比

开源工具名称	核心设计思路	内置特色能力	适用场景
SpecKite	SDD 规范驱动开发		新手快速落地规范化项目

开源工具名称	核心设计思路	内置特色能力	适用场景
		• 拆解需求生成规范文档 • 分阶段明确验收标准 • AI 严格按文档逐步开发	
Superpowers	Agent Skills 框架	• 内置完整开发工作流 • 强制 TDD 测试驱动开发 • 两阶段代码审查与子代理协作	需要标准化管理的项目体系

新手落地标准 5 步法：

- 1. 前置规则配置： 编写 AGENTS.md 明确项目背景、技术栈与代码规范。
- 2. 阶段管控要求： 严格遵循“先确认方案再启动编码”的铁律。
- 3. 工具配套能力： 挂载 MCP 与 Skills 工具，赋予 AI 检索最新技术资料的能力。
- 4. AI 自测与纠偏： 依靠 Cursor browser use 等自主测试，卡点时人工介入微调线索。
- 5. 沉淀与提交： 每阶段更新架构文档并提交 Git，建立完整的回滚检查点。